



NEW YORK INSTITUTE OF TECHNOLOGY

College of Engineering
& Computing Sciences

COURSE NAME: COMPUTATIONAL ANALYSIS

COURSE CODE: MATH330-M01

SEMESTER: SPRING 2026

Project Four

Least Squares Fitting

AUTHOR: JASMINE TUIACHIEVA

INSTRUCTOR: DR. ANDREW HOFSTRAND

SUBMISSION DATE: 03-03-2026 (MM-DD-YYYY)

TABLE OF CONTENTS

Abstract.....	2
Concept	2
Defining the Distance	2
Linear Least Squares Solution.....	3
The Unique Minimizer and Cauchy-Schwarz	3
Linear Implementation	5
Quadratic Implementation	7
Calculus Derivation of the 3x3 System.....	7
Mathematica Implementation.....	7
Matrix Algebra	8
Conclusion	9
Acknowledgements.....	9
References	10
Primary Course Documentation.....	10
Data Sources	10
Mathematical Theories & Encyclopedic Resources	10
Software & Wolfram Language Documentation.....	10

ABSTRACT

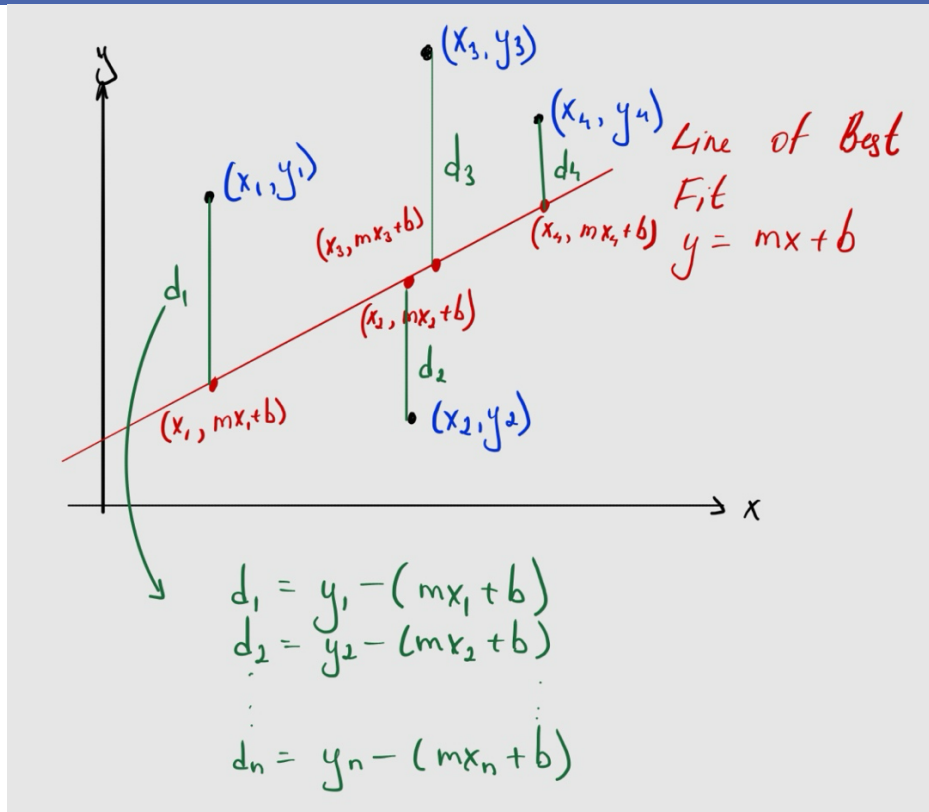


FIGURE 1. Least Squares Method using a Line of Best Fit $y = mx + b$

Least Squares Fitting is a mathematical procedure for finding the **best-fitting curve** to a given set of data points by **minimizing** the **sum of the squares** of the vertical deviations (FIGURE 1) between each data point and the curve. In this project, we transition from a simple **linear model** to a **quadratic model** to better capture the inherent trends in the provided dataset.

CONCEPT

DEFINING THE DISTANCE

Given a set of n data points $X_n = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_n, y_n)\}$, we aim to fit the data with a model (starting with a line $y = mx + b$). This is an **optimization** problem where we must decide which **distance to minimize**. As outlined in the course notes, we consider three possibilities, where D is the **function of m (slope) and b (y-intercept)**, since these are the only values that vary:

1. $D_1(m, b) = \sum_{j=1}^n (y_j - (mx_j + b))$
2. $D_2(m, b) = \sum_{j=1}^n |y_j - (mx_j + b)|$
3. $D_3(m, b) = \sum_{j=1}^n (y_j - (mx_j + b))^2$

D_1 is not well suited because errors can be **negative** (e. g. when we have a data point below the line of best fit) and cancel each other out. D_2 uses absolute values, making it **non-differentiable**. We choose D_3 (labeled simply as D) because it is always **non-negative and differentiable**, allowing us to use calculus to find the minimum. This is the origin of the term "**Least Squares**."

LINEAR LEAST SQUARES SOLUTION

We find the **first derivatives** to locate the Critical Points. In calculus, a horizontal tangent (where the derivative is zero) indicates a **potential minimum, maximum, or saddle point**. By setting partial derivatives to zero, we find the specific values m, b of where the error function stops changing. After differentiating D and setting the first-order partial derivatives to zero, we have:

$$\frac{\partial D}{\partial m} = -2 \sum_{j=1}^n (y_j - mx_j - b)x_j = 0$$

$$\frac{\partial D}{\partial b} = -2 \sum_{j=1}^n (y_j - mx_j - b) = 0$$

Rearranging these into matrix form:

$$\begin{bmatrix} \sum x_j^2 & \sum x_j \\ \sum x_j & n \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} = \begin{bmatrix} \sum x_j y_j \\ \sum y_j \end{bmatrix}$$

THE UNIQUE MINIMIZER AND CAUCHY-SCHWARZ

Finding the first derivative only tells us the surface is "**flat**" at that point. It doesn't guarantee we are at the bottom of a bowl (minimum) rather than the top of a hill (maximum). In single-variable calculus, we use the **Second Derivative Test**: if $f''(x) > 0$, the function is concave up and we have a minimum. In multi-variable calculus, we have two variables (m, b) so we use the **Hessian Matrix** (the matrix of second partial derivatives) to perform the same test.

In our case the Hessian Matrix is:

$$H(D) = 2 \begin{bmatrix} \sum x_j^2 & \sum x_j \\ \sum x_j & n \end{bmatrix}$$

For a surface to be “**concave up**” in **3D**, it must be curving upwards in **every direction**.

- If the **determinant** of the Hessian is positive, it means the eigenvalues are either both positive (concave up) or both negative (concave down).
- By checking that the **diagonal elements** (like $2 \sum x^2$ or $2n$) are also positive, we confirm the surface is curving upwards like a bowl.

If the determinant were **negative**, the surface would curve up in one direction and down in another, creating a saddle point (like a Pringles chip, **FIGURE 2**), where no unique “best fit” exists.

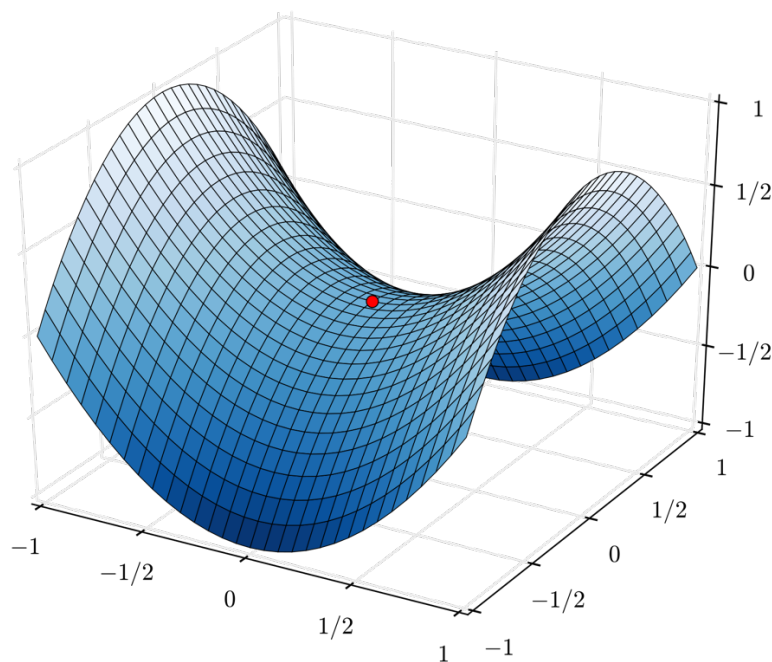


FIGURE 2. A saddle point (in red) on the graph of $z = x^2 - y^2$ (hyperbolic paraboloid)

The determinant is:

$$\det(H(D)) = 4 \left[n \sum x_j^2 - \left(\sum x_j \right)^2 \right]$$

Setting the determinant bigger than zero we have:

$$4 \left[n \sum x_j^2 - \left(\sum x_j \right)^2 \right] > 0$$

$$\left[n \sum x_j^2 - \left(\sum x_j \right)^2 \right] > 0$$

$$n \sum x_j^2 > \left(\sum x_j \right)^2$$

$$\left(\sum x_j \right)^2 < n \sum x_j^2$$

To verify this is true we use **Cauchy-Schwarz Theorem**, which states that given the vectors $\mathbf{u} = (u_1, \dots, u_n)$ and $\mathbf{v} = (v_1, \dots, v_n)$ in $\mathbb{R}^n$, we have:

$$\mathbf{u} \cdot \mathbf{v} \leq |\mathbf{u}| |\mathbf{v}|$$

In our case, let $\mathbf{u} = (x_1, \dots, x_n)$ and $\mathbf{v} = (1, \dots, 1)$, then $\mathbf{u} \cdot \mathbf{v} \leq |\mathbf{u}| |\mathbf{v}|$ is:

$$\mathbf{u} \cdot \mathbf{v} \leq |\mathbf{u}| |\mathbf{v}|$$

$$(x_1, \dots, x_n) \cdot (1, \dots, 1) \leq \sqrt{(x_1^2 + x_2^2 + \dots + x_n^2)} \sqrt{(1^2 + 1^2 + \dots + 1^2)}$$

$$\sum x_j \leq \sqrt{n} \sqrt{\sum x_j^2}$$

Squaring both sides, we get:

$$\left(\sum x_j \right)^2 < n \sum x_j^2$$

This means that the determinant of the Hessian is nearly always positive, proving we have found a unique global minimum.

LINEAR IMPLEMENTATION

Let's implement the linear model on the data set from **FIGURE 3** on Mathematica.

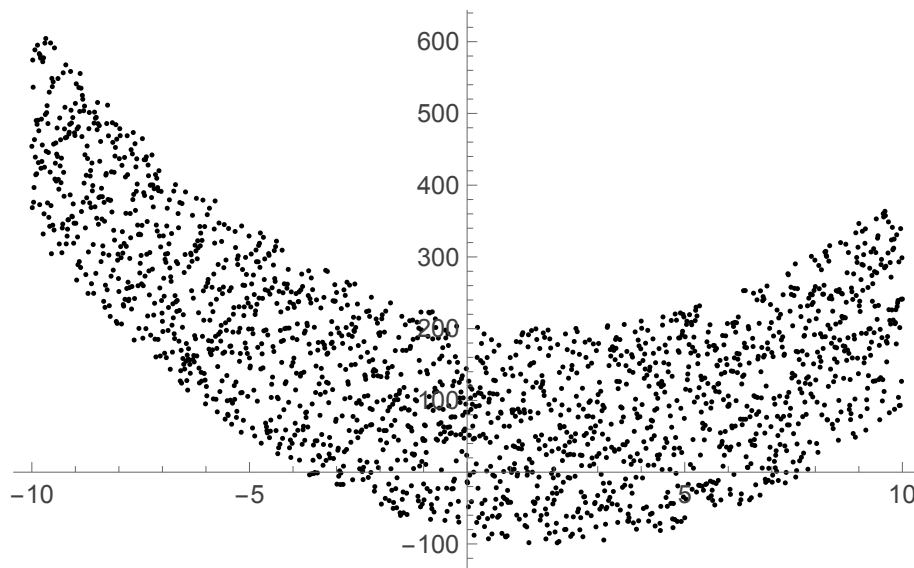


FIGURE 3. The plot of n data points $X_n = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_n, y_n)\}$

We start by loading the data from the .txt file and computing all required sums to construct the matrix M:

```
(*Loading the data*)
data = ToExpression[
  Import["/Users/jasminet/Downloads/data_leastqs.txt", "List"]];
xVal = data[[All, 1]];
yVal = data[[All, 2]];
n = Length[data];

(*Calculating Sums for the 2x2 Linear System from Equation 35*)
sumX = Sum[xVal[[i]], {i, 1, n}];
sumY = Sum[yVal[[i]], {i, 1, n}];
sumX2 = Sum[xVal[[i]]^2, {i, 1, n}];
sumXY = Sum[xVal[[i]]*yVal[[i]], {i, 1, n}];
```

Now, we construct the matrix, solve for coefficients and eventually plot the result:

```
(*Solving M . coefficientsMatrix = v*)
mLin = {{sumX2, sumX}, {sumX, n}};
vLin = {sumXY, sumY};
coeffsLin = Inverse[mLin] . vLin;
{m, b} = coeffsLin;

(*Plotting the Line of Best Fit*)
pointsPlot = ListPlot[data, PlotStyle -> Black];
linePlot =
  Plot[m*x + b, {x, Min[xVal], Max[xVal]}, PlotStyle -> {Red, Thick}];
Show[pointsPlot, linePlot, PlotLabel -> "Line of Best Fit"]
```

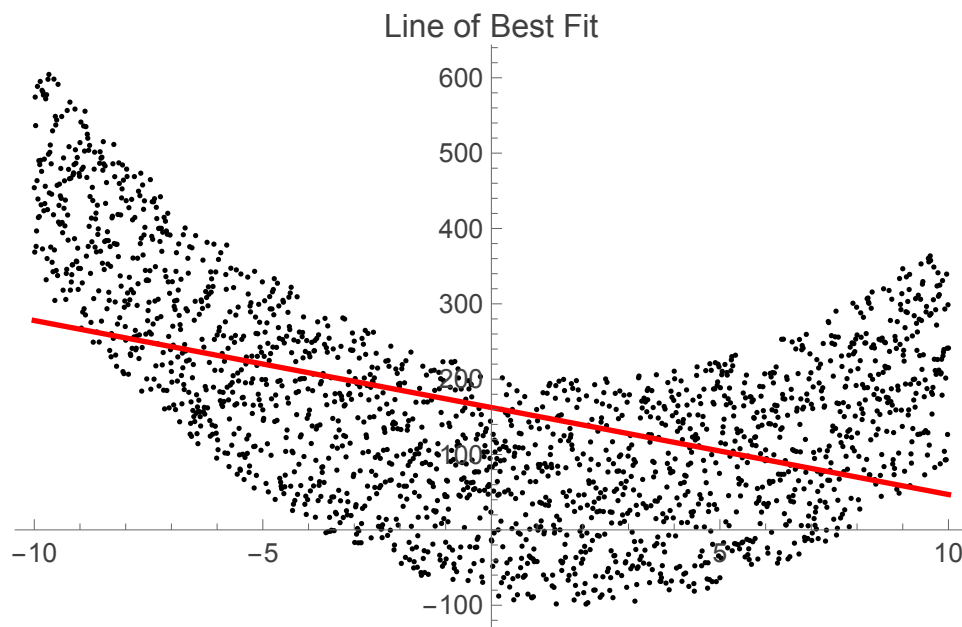


FIGURE 4. The computed Line of Best fit in **red** on a sample data set in **black**.

QUADRATIC IMPLEMENTATION

The data from **FIGURE 3** exhibits a clear curve that a line cannot mimic. Let's generalize the concept of *line of best fit* to the *parabola of best fit*, since it is still a regression problem.

CALCULUS DERIVATION OF THE 3X3 SYSTEM

We start by defining the Distance function D or the Error function:

$$D(a, b, c) = \sum_{j=1}^n (y_j - (ax_j^2 + bx_j + c))^2$$

Differentiating the above equation and setting it to zero, we find the partial derivatives:

- $\frac{\partial D}{\partial a} = \sum 2(y_j - ax_j^2 - bx_j - c)(-x_j^2) = 0$
- $\frac{\partial D}{\partial b} = \sum 2(y_j - ax_j^2 - bx_j - c)(-x_j) = 0$
- $\frac{\partial D}{\partial c} = \sum 2(y_j - ax_j^2 - bx_j - c)(-1) = 0$

The Resulting Matrix M is:

$$\begin{bmatrix} \sum x_j^4 & \sum x_j^3 & \sum x_j^2 \\ \sum x_j^3 & \sum x_j^2 & \sum x_j \\ \sum x_j^2 & \sum x_j & n \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} \sum x_j^2 y_j \\ \sum x_j y_j \\ \sum y_j \end{bmatrix}$$

MATHEMATICA IMPLEMENTATION

Now, we implement the derived model into the same data set.

Let's start by calculating the higher-order sums that haven't been used before:

```
(*Calculating higher order Sums for the 3x3 System*)
sumX3 = Sum[xVal[[i]]^3, {i, 1, n}];
sumX4 = Sum[xVal[[i]]^4, {i, 1, n}];
sumX2Y = Sum[xVal[[i]]^2*yVal[[i]], {i, 1, n}];
```

Continue by constructing the matrix according to the matrix M we found above and solving for coefficients a, b, c :

```
*Constructing Matrix M and Vector v*)
M = {{sumX4, sumX3, sumX2}, {sumX3, sumX2, sumX}, {sumX2, sumX, n}};
v = {sumX2Y, sumXY, sumY};

(*Solving for coefficients a,b,c*)
coeffsQuad = Inverse[M] . v;
```

```
{a, bQuad, c} = coeffsQuad; (*b coefficient named bQuad to prevent variable
name conflicts with b variable from the Line of Best Fit*)
```

Finally, construct the parabola equation using the found values for a , b , c , plot the data set and parabola of best fit:

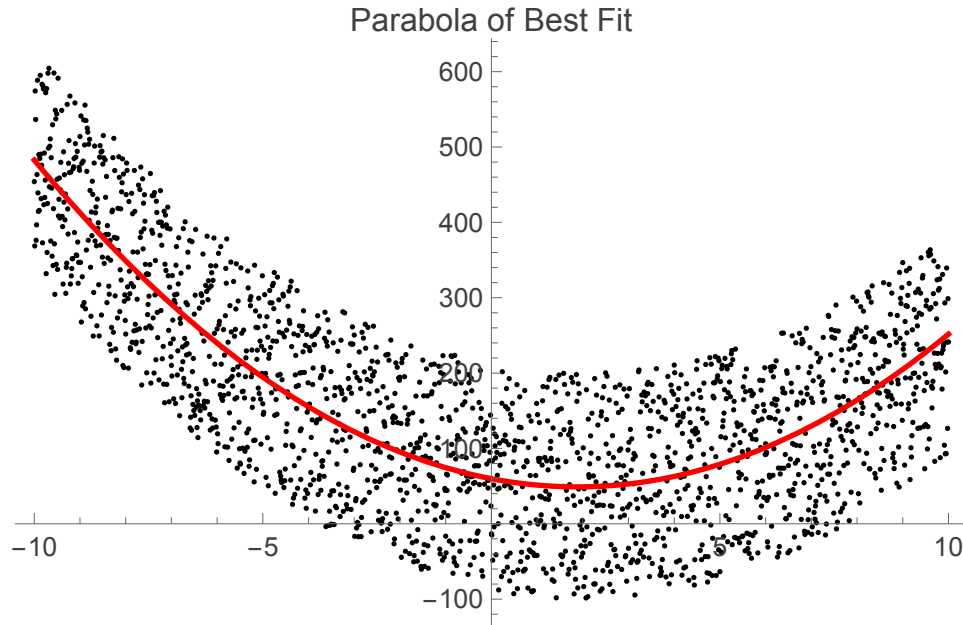


FIGURE 5. The computed Parabola of Best fit in **red** on a sample data set in **black**.

MATRIX ALGEBRA

If we have n data points, we want our parabola $y = ax^2 + bx + c$ to get as close as possible to every point (x_j, y_j) . For each point, we can write an equation:

Point 1 : $ax_1^2 + bx_1 + c \approx y_1$

Point 2 : $ax_2^2 + bx_2 + c \approx y_2$

...

Point n : $ax_n^2 + bx_n + c \approx y_n$

We can rewrite this entire stack of n equations as a single matrix multiplication:

$$\underbrace{\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix}}_A \begin{bmatrix} a \\ b \\ c \end{bmatrix} \approx \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

This matrix A has **n rows** (one for each data point) and **3 columns** (one for each variable a , b , c).

But, if we transpose of A , A^T , and multiply it by A :

$$A^T A = \begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix} \begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix} = \begin{bmatrix} \sum x_j^4 & \sum x_j^3 & \sum x_j^2 \\ \sum x_j^3 & \sum x_j^2 & \sum x_j \\ \sum x_j^2 & \sum x_j & n \end{bmatrix} = M$$

Moreover, since we know the right-hand side of our normal equations is $v = \begin{bmatrix} \sum x_j^2 y_j \\ \sum x_j y_j \\ \sum y_j \end{bmatrix}$, we

can see that:

$$v = A^T y = \begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

So, **B is simply A^T** . And for any vector q :

$$q^T M q = q^T (A^T A) q = (Aq)^T (Aq) = \|Aq\|^2$$

Since the squared norm $\|Aq\|^2 \geq 0$, M is **positive semi-definite**. This confirms the **error surface is a convex bowl with a unique global minimum**.

CONCLUSION

Generalizing the least squares problem to an m -th degree polynomial involves an $n \times (m + 1)$ matrix A . When $m = n - 1$, the least squares fit becomes an **interpolation**, passing exactly through every point. This project demonstrates that while the resulting model is **non-linear**, the process of finding the *coefficients remains a linear optimization problem*.

ACKNOWLEDGEMENTS

I would like to express my gratitude to Dr. Andrew Hofstrand for his guidance and for providing the theoretical framework necessary to complete this project.

REFERENCES

PRIMARY COURSE DOCUMENTATION

Hofstrand, A. (2026). MATH330: *Computational Analysis* – Section 6.1-6.2: Least Squares Fitting. New York Institute of Technology.

DATA SOURCES

NYIT Department of Mathematics. (2026). data_leastqs.txt. [Dataset containing coordinate pairs for **FIGURE 3**.].

MATHEMATICAL THEORIES & ENCYCLOPEDIC RESOURCES

Wikipedia. Cauchy-Schwarz Inequality. https://en.wikipedia.org/wiki/Cauchy-Schwarz_inequality

Wikipedia. (2026). Saddle Point. https://en.wikipedia.org/wiki/Saddle_point

SOFTWARE & WOLFRAM LANGUAGE DOCUMENTATION

Wolfram Research. (n.d.). Array. Wolfram Language & System Reference Guide. <https://reference.wolfram.com/language/ref/Array.html>

Wolfram Research. (n.d.). Dot Product. Wolfram Language & System Reference Guide. <https://reference.wolfram.com/language/ref/Dot.html>

Wolfram Research. (n.d.). Linear Algebra Tutorial. Wolfram Language & System Reference Guide. <https://reference.wolfram.com/language/tutorial/LinearAlgebra.html>

Wolfram Research. (n.d.). MatrixForm. Wolfram Language & System Reference Guide. <https://reference.wolfram.com/language/ref/MatrixForm.html>

Wolfram Research. (n.d.). Transpose. Wolfram Language & System Reference Guide. <https://reference.wolfram.com/language/ref/Transpose.html>